

LLVM & COMPILER ENGINEERING

LLVM ile Oyun Programlama Dili Geliştirme

Modern C++17/20, Frontend AST, LLVM IR, Pass Manager,
CodeGen & ORC JIT Rehberi

Geliştirici & Eğitim Kılavuzu
Kapsamlı Mimari ve Uygulama Kitabı

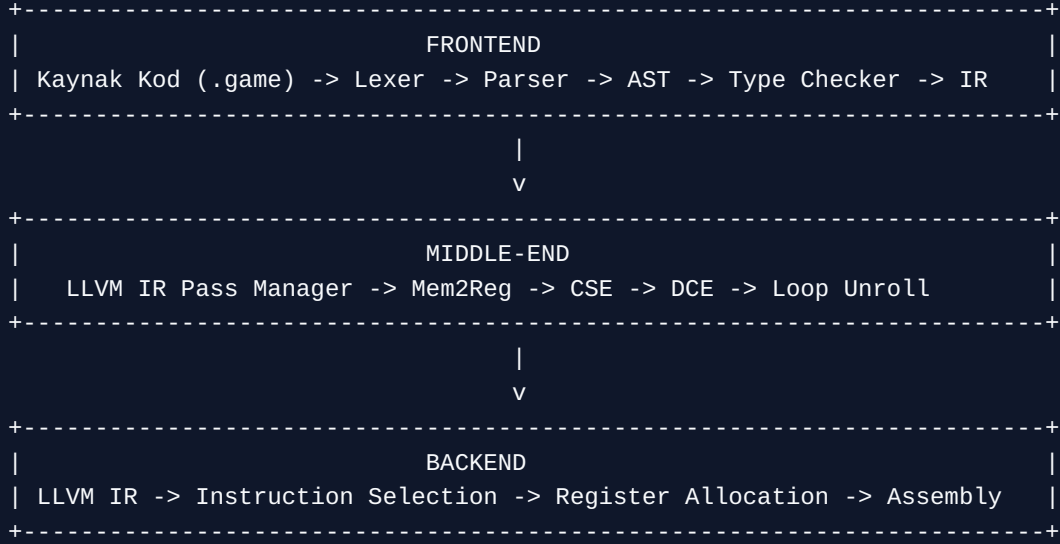
Bölüm 1: LLVM Mimarisine Giriş, Derleyici Mimarileri ve Oyun Motoru Dilleri İçin LLVM Ortam Kurulumu

1.1 Derleyici Mimarilerine Genel Bakış: Klasik vs. Modern Üç Aşamalı (Three-Phase) Yapı

Bir programlama dilinin kaynak kodunu (source code) hedef makinenin anlayabileceği makine koduna (machine code) dönüştürme süreci, bilgisayar bilimlerinin en karmaşık ve zarif alanlarından biridir. Geleneksel (monolitik) derleyicilerde, kaynak dilden hedef mimariye doğrudan bir geçiş yapılırdı. Bu durum, N adet programlama dili ve M adet hedef mimari (x86, ARM, RISC-V, WebAssembly vb.) olduğunda $N \times M$ adet farklı derleyici yazılmasını gerektiriyordu.

Modern derleyici tasarımında bu problem, **Üç Aşamalı Mimariler (Three-Phase Compiler Architecture)** ile çözülmüştür. Bu mimari üç ana bileşenden oluşur:

- Frontend (Ön Yüz):** Kaynak kodu okur, sözdizimsel (lexical) ve dilbilgisel (syntactic) analizini yapar, Tip Denetimi (Type Checking) gerçekleştirir ve dili temsil eden bir **Soyut Sözdizim Ağacı (Abstract Syntax Tree - AST)** oluşturur. Son olarak bu AST'yi dile bağımsız bir **Ara Temsile (Intermediate Representation - IR)** dönüştürür.
- Middle-end (Orta Yüz / Optimizasyon Katmanı):** Üretilen IR üzerinde çalışır. Kaynak dilden ve hedef donanımdan bağımsız olarak kod üzerindeki gereksiz hesaplamaları ayıklar, döngüleri optimize eder, bellek erişimlerini iyileştirir ve kodu en verimli hale getirir.
- Backend (Arka Yüz / Kod Üretimi):** Optimize edilmiş IR'ı alır, hedef mimarinin yazmaç (register) kısıtlarına, buyruk kümesine (instruction set) ve yürütme boru hattına (pipeline) göre makine koduna (assembly / makine komutları) dönüştürür.



Bu üç aşamalı yaklaşım sayesinde N dil ve M mimari için ihtiyaç duyulan dönüşüm karmaşıklığı $N + M$ seviyesine inmiştir. LLVM, tam olarak bu mimarinin kalbinde yer alır.

Derleyici Mimarisinde Katman Ayrımı

Frontend dilden anlar fakat işlemciden anlamaz; Backend işlemciden anlar fakat dilden anlamaz; Middle-end ise ne dilden ne de işlemciden anlar, sadece evrensel LLVM IR üzerindeki matematiksel ve mantıksal optimizasyonlara odaklanır.

1.2 LLVM Felsefesi ve Ekosistemi

LLVM (kökensel olarak *Low Level Virtual Machine*, ancak günümüzde sadece bir marka isimdir), Chris Lattner ve Vikram Adve tarafından Illinois Üniversitesi'nde başlatılan açık kaynaklı bir derleyici altyapısı projesidir. LLVM'in temel felsefesi **Modülerlik**, **Yeniden Kullanılabilirlik** ve **Dil/Mimari Bağımsızlığıdır**.

LLVM Ekosistemi tek bir derleyiciden ibaret değildir; birbiriyle entegre çalışan devasa bir araçlar ve kütüphaneler koleksiyonudur:

- **LLVM Core:** Temel kod optimizasyonu ve her türlü işlemci mimarisi için kod üretimi sağlayan C++ kütüphaneleri koleksiyonu.
- **Clang:** C, C++, Objective-C ve OpenCL için geliştirilmiş, LLVM tabanlı ultra hızlı bir Frontend derleyicisi.
- **LLD:** LLVM projesinin geliştirdiği, GNU gold veya MSVC link.exe'ye kıyasla katlarca hızlı çalışan sistem bağlayıcısı (Linker).
- **LLDB:** LLVM mimarisi üzerine kurulu yüksek performanslı hata ayıklayıcı (Debugger).

- **MLIR (Multi-Level Intermediate Representation):** Etki alanına özel (Domain-Specific) derleyiciler ve yapay zeka/makine öğrenmesi modelleri için çok seviyeli ara temsil altyapısı.
- **LLVM ORC JIT:** Çalışma zamanında (JIT - Just-In-Time) dinamik kod derlemeyi sağlayan modüler C++ API'si.

1.3 Oyun Motorları İçin Özel Programlama Dili Geliştirme İhtiyacı

Oyun geliştirme; yüksek performans (FPS kısıtları), düşük gecikme (latency), esnek betik yazımı (scripting) ve GPU/CPU arasında kesintisiz haberleşme gerektiren zorlu bir alandır. C++ geleneksel olarak sektör standardı olsa da, derleme sürelerinin uzunluğu ve esnek olmayan yapısı nedeniyle oyun projelerinde sıklıkla betik dilleri (Lua, C#, Cscript vb.) kullanılır.

Ancak Lua gibi yorumlanan (interpreted) diller çalışma zamanında ciddi performans kayıplarına yol açar. Oyun motorları için özel bir dil oluştururken LLVM kullanmanın kritik avantajları şunlardır:

1. **Sıfır Maliyetli Soyutlama (Zero-Cost Abstraction):** LLVM'in güçlü optimizasyon katmanı sayesinde dilde yazdığınız yüksek seviyeli yapılar (örneğin 3D Vektör toplama, ECS bileşen erişimleri) doğrudan inline C++ seviyesinde makine koduna dönüşür.
2. **JIT Scripting Hibrit Yapısı:** Oyun oynanırken betik kodlarını çalışma zamanında LLVM ORC JIT ile makine koduna derleyebilir, interpret etme yavaşlığından tamamen kurtulabilirsiniz.
3. **GPU Shader ve CPU Kod Bütünlüğü:** LLVM SPIR-V veya DirectX Shader Compiler (DXC) altyapısı sayesinde oyun dilinizden doğrudan Vulkan/DirectX shader kodları üretebilirsiniz.
4. **Veri Odaklı Tasarım (Data-Oriented Design) Desteği:** Bellek düzenini (SIMD alignment, Struct-of-Arrays) LLVM IR seviyesinde milimetrik olarak kontrol edebilirsiniz.

1.4 Oyun Motoru C++ Geliştirme Ortamı ve LLVM Kurulumu

Oyun dilinizi yazarken LLVM C++ kütüphanelerini projenize bağlamanız gerekir. Modern C++ (C++17 / C++20) standartları ile LLVM API'lerini kullanacağız.

CMake Entegrasyonu (`CMakeLists.txt`)

Aşağıda, sisteminizde kurulu olan LLVM paketini otomatik tespit edip C++ projenize bağlayan profesyonel bir `CMakeLists.txt` konfigürasyonu yer almaktadır:

```

cmake_minimum_required(VERSION 3.20)
project(GameLangCompiler LANGUAGES CXX C)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

# LLVM paketini sistemde ara
find_package(LLVM REQUIRED CONFIG)

message(STATUS "LLVM Bulundu: sürüm ${LLVM_PACKAGE_VERSION}")
message(STATUS "LLVM Dahil Etme Dizini: ${LLVM_INCLUDE_DIRS}")
message(STATUS "LLVM Tanımlamaları: ${LLVM_DEFINITIONS}")

# LLVM Başlık Dosyalarını Ekle
include_directories(${LLVM_INCLUDE_DIRS})
add_definitions(${LLVM_DEFINITIONS})

# Oyun Derleyicisi Çalıştırılabilir Dosyası
add_executable(gamelang_compiler
    src/main.cpp
)

# Gerekli LLVM Bileşenlerini/Kütüphanelerini Map'le
llvm_map_components_to_libnames(llvm_libs
    Core
    Analysis
    ExecutionEngine
    InstCombine
    Object
    OrcJIT
    ScalarOpts
    Support
    TransformUtils
    native
)

# Kütüphaneleri Hedefe Bağla
target_link_libraries(gamelang_compiler PRIVATE ${llvm_libs})

```

1.5 İlk LLVM C++ Programı: LLVMContext , Module , ve IRBuilder

Kavramları

LLVM C++ API'sinde çalışırken bilmeniz gereken üç temel yapı vardır:

1. **llvm::LLVMContext** : LLVM'in tüm durum bilgilerini (tip tabloları, sabitler, çekirdek veri yapıları) yöneten ipucu nesnesidir. İki farklı thread üzerinde çalışırken her thread kendi LLVMContext nesnesine sahip olmalıdır.
2. **llvm::Module** : Bir derleme birimini (Translation Unit) temsil eden ana konteynerdir. İçerisinde fonksiyonlar, küresel değişkenler, tip tanımları ve hedef mimari bilgileri barındırır.

3. `llvm::IRBuilder<>` : Basic Block'lar içerisine adım adım LLVM IR talimatı (instruction) ekleyen, tür güvenli yardımcı C++ şablon sınıfıdır.

Detaylı C++ Örneği: Oyun Vektör Toplama Fonksiyonunun LLVM IR Olarak Üretilmesi (`src/main.cpp`)

Aşağıdaki C++ kodu, oyun motorlarında sıkça kullanılan iki boyutlu bir vektör toplama fonksiyonunu (`Vec2Add`) LLVM C++ API'sini kullanarak hafızada oluşturur:

```

#include <llvm/IR/LLVMContext.h>
#include <llvm/IR/Module.h>
#include <llvm/IR/IRBuilder.h>
#include <llvm/IR/Verifier.h>
#include <llvm/Support/raw_ostream.h>
#include <iostream>
#include <memory>
#include <vector>

int main() {
    // 1. LLVM Context Oluşturma
    auto context = std::make_unique<llvm::LLVMContext>();

    // 2. Oyun Dili Modülü Oluşturma
    auto module = std::make_unique<llvm::Module>("GameEngineMathModule", *context);

    // 3. IR Builder Nesnesi
    llvm::IRBuilder<> builder(*context);

    // 4. Vector2D Toplama Fonksiyonu İmzası Oluşturma: float Vec2Add(float x1, float y1, float x2, float y2)
    // Parametre tipleri: [float, float, float, float]
    llvm::Type* floatType = builder.getFloatTy();
    std::vector<llvm::Type*> paramTypes = {floatType, floatType, floatType, floatType};

    // Fonksiyon Tipi: Geri dönüş float, parametreler float vector bileşenleri
    llvm::FunctionType* funcType = llvm::FunctionType::get(floatType, paramTypes, false);

    // Fonksiyonu Modüle Ekleme (External Linkage)
    llvm::Function* vec2AddFunc = llvm::Function::Create(
        funcType,
        llvm::Function::ExternalLinkage,
        "Vec2Add",
        module.get()
    );

    // Parametre İsimlerini Belirleme
    auto argIt = vec2AddFunc->arg_begin();
    llvm::Value* x1 = argIt++; x1->setName("x1");
    llvm::Value* y1 = argIt++; y1->setName("y1");
    llvm::Value* x2 = argIt++; x2->setName("x2");
    llvm::Value* y2 = argIt++; y2->setName("y2");

    // 5. Basic Block (Giriş Bloğu) Oluşturma
    llvm::BasicBlock* entryBB = llvm::BasicBlock::Create(*context, "entry", vec2AddFunc);
    builder.SetInsertPoint(entryBB);

    // 6. İşlemler: x_sum = x1 + x2; y_sum = y1 + y2; result = x_sum + y_sum
    llvm::Value* xSum = builder.CreateFAdd(x1, x2, "x_sum");
    llvm::Value* ySum = builder.CreateFAdd(y1, y2, "y_sum");
    llvm::Value* totalSum = builder.CreateFAdd(xSum, ySum, "total_sum");

    // Fonksiyondan Dönen Değer
    builder.CreateRet(totalSum);
}

```

```
// 7. Oluşturulan Fonksiyonun Doğruluğunu (Validation) Kontrol Etme
std::string errorStr;
llvm::raw_string_error_stream errorStream(errorStr);
if (llvm::verifyFunction(*vec2AddFunc, &llvm::outs())) {
    llvm::errs() << "HATA: Vec2Add fonksiyonu LLVM IR kurallarına uymuyor!\n";
    return 1;
}

// 8. Modülün IR Çıktısını Ekrana Yazdırma
llvm::outs() << "=== Oyun Dili Vec2Add LLVM IR Ciktisi ===\n";
module.print(llvm::outs(), nullptr);

return 0;
}
```

İpucu: LLVM IR Doğrulama (llvm::verifyFunction)

`llvm::verifyFunction` çağırısı, oluşturduğunuz IR'ın LLVM tip sistemine ve SSA kurallarına uyup uymadığını denetler. Derleyici geliştirirken bu adımı atlamamak, beklenmeyen çalışma zamanı çökmelerini önler.

Koda Adım Adım Bakış ve Açıklaması:

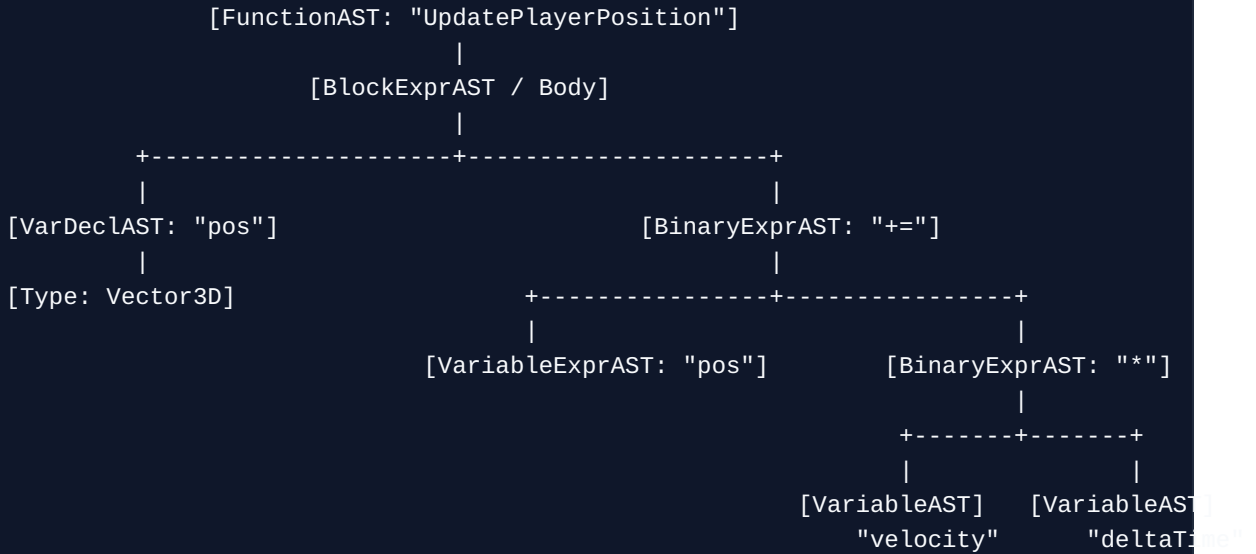
1. **getFloatTy()** : LLVM Context üzerinden tek duyarlıklı kayan noktalı sayı (`float`) tipini alır.
2. **FunctionType::get** : Fonksiyonun geri dönüş tipini ve alacağı parametre listesini tanımlayan metatipi üretir.
3. **Function::Create** : Oluşturulan fonksiyonu `GameEngineMathModule` içerisine yerleştirir.
4. **BasicBlock::Create** : Fonksiyonun kod yürütme adımlarının tutulacağı ilk Temel Bloğu (`entry`) tanımlar.
5. **builder.CreateFAdd** : Kayan noktalı sayılar için LLVM `fadd` talimatını üretir.

Bölüm 2: Frontend Tasarımı, AST, LLVM IR Yapısı ve Bellek Modeli

2.1 Soyut Sözdizim Ağacı (AST) Tasarımı

Ön Yüz (Frontend) mimarisinin kalbinde **Soyut Sözdizim Ağacı (Abstract Syntax Tree - AST)** yer alır. AST, kaynak kodun dilbilgisel yapısını düğümler (nodes) halinde temsil eden hiyerarşik bir ağaç veri yapısıdır. Oyun programlama dilimizde hem ifade (expression) hem de bildirim/deyim (statement) düğümlerine ihtiyacımız vardır.

Modern C++ (C++17/20) standartlarında AST düğümlerini `std::unique_ptr` ve sanal fonksiyonlar (polymorphism) veya `std::variant` ile temsil ederiz. Her AST düğümü, kendini LLVM IR'a dönüştürme sorumluluğuna (`codegen()`) sahiptir.



2.2 LLVM IR (Intermediate Representation) Derinlemesine İnceleme

LLVM IR, LLVM derleyici altyapısının merkezindeki ortak dildir. Üç farklı biçimde temsil edilebilir: 1. **Hafızadaki (In-Memory) Veri Yapısı:** C++ sınıfları (`llvm::Instruction`, `llvm::Value`, `llvm::BasicBlock`). 2. **Disk Üzerindeki Okunabilir Metin Biçimi (.ll):** İnsanlar tarafından kolayca okunabilen montaj dili benzeri yapı. 3. **Disk Üzerindeki İkili Bitcode Biçimi (.bc):** Derleyicilerin hızlı işleyebilmesi için optimize edilmiş ikili format.

LLVM IR, güçlü bir şekilde tiplendirilmiş (strongly typed), sonsuz sayıda sanal yazmaca (virtual register) sahip ve **Tekli Atama Biçimine (Static Single Assignment - SSA)** dayalı bir yapıdır.

2.3 Tekli Atama Biçimi (Static Single Assignment - SSA) ve PHI Düğümleri

SSA Nedir?

SSA formunda, her değişken **yalnızca bir kez** atanabilir. Klasik bir programlama dilinde bir değişkene birden fazla kez değer atanabilir:

```
// Klasik C/C++ Kodu
int health = 100;
health = health - damage;
health = health + healAmount;
```

Bu kod SSA yapısına dönüştürüldüğünde, her atama yeni bir SSA sürüm değişkeni (register) oluşturur:

```
; LLVM IR SSA Biçimi
%health0 = copy i32 100
%health1 = sub i32 %health0, %damage
%health2 = add i32 %health1, %healAmount
```

SSA formunun derleyicilere sağladığı en büyük avantaj, veri akışı analizini (Data Flow Analysis) inanılmaz derecede basitleştirmesidir. Bir yazmacın değerinin nereden geldiği ve nerede değiştiği tartışmasız bir şekilde bellidir.

PHI Düğümleri (`phi` instruction)

Dallanma (if-else, döngüler) olan durumlarda bir değişkenin hangi kontrol akışı yolundan geldiği çalışma zamanında belli olur. SSA kuralını bozmadan bu durumu çözmek için **PHI Düğümleri (`phi` instruction)** kullanılır.

```

; Oyuncu Can Durumu Örneği
entry:
    %cmp = icmp sgt i32 %damage, %shield
    br i1 %cmp, label %take_damage, label %blocked

take_damage:
    %hp_after_hit = sub i32 %current_hp, %damage
    br label %merge

blocked:
    %hp_after_block = select i1 true, i32 %current_hp, i32 0
    br label %merge

merge:
    ; PHI Döğümü: Kodun hangi bloktan geldiğine göre doğru hp değerini seçer
    %final_hp = phi i32 [ %hp_after_hit, %take_damage ], [ %hp_after_block, %blocked ]
    ret i32 %final_hp

```

PHI Döğümü Kuralı

LLVM IR içerisinde bir PHI döğümü talimatı (``phi``), bulunacağı Basic Block'un ****en başında**** (diğer tüm işlem talimatlarından önce) yer almak zorundadır!

2.4 Temel Yapı Taşları (Basic Blocks) ve Kontrol Akış Grafiğı (CFG)

Bir **Temel Yapı Taşı (Basic Block)**, içerisinde yalnızca tek bir noktadan girilen (ilk talimat) ve yalnızca tek bir noktadan çıkılan (son talimat) talimatlar dizisidir.

Bir Basic Block'un son talimatı mutlaka bir **Terminator Instruction** (Sonlandırıcı Talimat) olmalıdır. Örnek sonlandırıcılar: * `ret` : Fonksiyondan döner. * `br` : Başka bir Basic Block'a dallanır (koşullu veya koşulsuz). * `switch` : Çoklu dallanma yapar.

Basic Block'ların birbirine `br` (branch) talimatları ile bağlanmasıyla **Kontrol Akış Grafiğı (Control Flow Graph - CFG)** elde edilir. Derleyici optimizasyonları (döngü tespiti, ölü kod eleme vb.) bu CFG üzerinde yürütölür.

2.5 Bellek Modeli ve `GetElementPtr` (GEP)

LLVM IR'da iki temel bellek erişim mantığı vardır: 1. **Yazmaç Tabanlı (Register-based SSA)**: `add`, `sub`, `mul` gibi işlemler doğrudan sanal yazmaçlar üzerinde yürür. 2. **Bellek Tabanlı (Memory-based Alloca)**: Yerel değişkenler yığında (`alloca`) tutulur, `load` ile okunur, `store` ile yazılır.

Yığın Tahsisi (`alloca`)

SSA formunu doğrudan elle yönetmek zor olduğu için ön yüz derleyicileri genellikle tüm yerel değişkenleri yığında `alloca` ile oluşturur.

```
%player_health = alloca i32, align 4
store i32 100, i32* %player_health, align 4
%val = load i32, i32* %player_health, align 4
```

LLVM'in `mem2reg` optimizasyon passı (Pass Manager), bu `alloca` bellek erişimlerini otomatik olarak SSA yazmaçlarına ve PHI düğümlerine dönüştürür!

`GetElementPtr` (GEP) Talimatı

`GetElementPtr` (GEP), LLVM IR'in en kritik ve sıkça yanlış anlaşılan talimatlarından biridir. GEP **belleğe erişmez (load/store yapmaz)**, yalnızca bellek adresini (pointer arithmetic) hesaplar!

Oyun motorumuzda bir `Entity` yapısının bellek düzenini düşünelim:

```
struct Entity {
    int id;           // 0. eleman (4 bayt)
    float health;     // 1. eleman (4 bayt)
    float position[3]; // 2. eleman (12 bayt: x, y, z)
};
```

LLVM IR üzerinde bir `Entity*` göstericisinden oyuncunun `position[1]` (y koordinatı) adresini hesaplamak için GEP şu şekilde kullanılır:

```
; %entity_ptr: Entity* türünde bir pointer
; 1. indis '0': Pointer'ın kendisi üzerinden dizide ilerleme (0 offset)
; 2. indis '2': Struct içindeki 2. alan (position dizisi)
; 3. indis '1': position dizisinin 1. elemanı (y koordinatı)

%y_ptr = getelementptr inbounds %struct.Entity, %struct.Entity* %entity_ptr, i32 0, i32 2, i32 1
%y_val = load float, float* %y_ptr, align 4
```

GEP Neden İlk İndis Olarak '0' Alır?

C/C++ dilindeki ``entity_ptr->position[1]`` ifadesinde, ``entity_ptr`` aslında bir dizi nesnesinin ilk elemanının adresidir (``*(entity_ptr + 0)``). İlk indis olan ``0``, pointer seviyesinde kaç eleman öteye gidileceğini belirler.

2.6 `IRBuilder<>` Kullanımı ve AST'den LLVM IR Üretimi

Şimdi bu kavramları bir araya getirerek oyun dilimiz için bir AST yapısı ve bu AST'yi LLVM IR'a dönüştüren C++ kodunu yazalım.

C++ Örneği: AST ve IR Generation (`src/frontend_ast.cpp`)

Aşağıdaki kapsamlı C++ örneğinde, bir oyun karakterinin canını ve zırhını güncelleyen `UpdateHealth` fonksiyonunun AST yapısı ve IR jenerasyonu gösterilmektedir:

```

#include <llvm/IR/LLVMContext.h>
#include <llvm/IR/Module.h>
#include <llvm/IR/IRBuilder.h>
#include <llvm/IR/Value.h>
#include <llvm/IR/Verifier.h>
#include <llvm/Support/raw_ostream.h>
#include <memory>
#include <string>
#include <vector>

// Taban AST Sınıfı
class ExprAST {
public:
    virtual ~ExprAST() = default;
    virtual llvm::Value* codegen(llvm::LLVMContext& context, llvm::Module& module, llvm::IRBuilder<>& builder) = 0;
};

// Sayısal Sabit AST Düğümü
class NumberExprAST : public ExprAST {
    double val;
public:
    NumberExprAST(double val) : val(val) {}

    llvm::Value* codegen(llvm::LLVMContext& context, llvm::Module& module, llvm::IRBuilder<>& builder) {
        return llvm::ConstantFP::get(context, llvm::APFloat(val));
    }
};

// Değişken Okuma AST Düğümü
class VariableExprAST : public ExprAST {
    std::string name;
public:
    VariableExprAST(const std::string& name) : name(name) {}

    llvm::Value* codegen(llvm::LLVMContext& context, llvm::Module& module, llvm::IRBuilder<>& builder) {
        // Not: Gerçek derleyicilerde Sembol Tablosundan (Symbol Table) okunur
        return nullptr;
    }
};

// İkili İşlem (Binary Expression: +, -, *) AST Düğümü
class BinaryExprAST : public ExprAST {
    char op;
    std::unique_ptr<ExprAST> lhs, rhs;
public:
    BinaryExprAST(char op, std::unique_ptr<ExprAST> lhs, std::unique_ptr<ExprAST> rhs) : op(op), lhs(std::move(lhs)), rhs(std::move(rhs)) {}

    llvm::Value* codegen(llvm::LLVMContext& context, llvm::Module& module, llvm::IRBuilder<>& builder) {
        llvm::Value* l = lhs->codegen(context, module, builder);
        llvm::Value* r = rhs->codegen(context, module, builder);
        if (!l || !r) return nullptr;

        switch (op) {

```

```

        case '+': return builder.CreateFAdd(l, r, "addtmp");
        case '-': return builder.CreateFSub(l, r, "subtmp");
        case '*': return builder.CreateFMul(l, r, "multmp");
        default: return nullptr;
    }
}
};

// Kullanım Örneği
void generateSampleAST() {
    llvm::LLVMContext context;
    llvm::Module module("GameASTModule", context);
    llvm::IRBuilder<> builder(context);

    // İfade: (10.0 + 20.0) * 2.5 (Örn: Oyun içi hasar çarpanı hesabı)
    auto expr = std::make_unique<BinaryExprAST>(
        '*',
        std::make_unique<BinaryExprAST>('+', std::make_unique<NumberExprAST>(10.0), std::make_unique<NumberExprAST>(20.0)),
        std::make_unique<NumberExprAST>(2.5)
    );

    // Dummy test fonksiyonu
    llvm::FunctionType* ft = llvm::FunctionType::get(builder.getDoubleTy(), false);
    llvm::Function* func = llvm::Function::Create(ft, llvm::Function::ExternalLinkage, "CalculateDamage", module);
    llvm::BasicBlock* bb = llvm::BasicBlock::Create(context, "entry", func);
    builder.SetInsertPoint(bb);

    llvm::Value* result = expr->codegen(context, module, builder);
    builder.CreateRet(result);

    llvm::verifyFunction(*func);
    module.print(llvm::outs(), nullptr);
}

```

Koda Adım Adım Bakış ve Açıklaması:

1. **ExprAST Arabirimi**: Tüm AST düğümlerinin tüzüğü olan saf sanal (pure virtual) base sınıftır.
2. **ConstantFP::get** : Çift duyarlıklı (double) sabit bir float LLVM **Value** nesnesi üretir.
3. **CreateFAdd / CreateFMul** : IEEE-754 uyumlu float toplama ve çarpma talimatlarını builder seviyesinde hafızaya yazar.
4. **builder.SetInsertPoint** : IRBuilder'ın yeni yazılacak olan talimatları hangi Basic Block içerisine ekleyeceğini belirler.

Bölüm 3: Middle-end Optimizasyonları ve New Pass Manager (NPM)

3.1 LLVM Optimizasyon Mimarisi ve Pass Kavramı

LLVM derleyici altyapısının en güçlü yönlerinden biri modüler **Pass (Geçiş)** mimarisidir. Bir **Pass**, LLVM IR üzerinde belirli bir analizi (Analysis Pass) yürüten veya IR'ı değiştiren/optimize eden (Transformation Pass) bağımsız bir C++ sınıfıdır.

Middle-end optimizasyon katmanı, kaynak dilden ve hedef donanımdan tamamen bağımsızdır. Amacı, ön yüzün ürettiği LLVM IR'ı matematiksel ve mantıksal olarak en sade, en hızlı ve en az bellek tüketen şekle getirmektir.

```

+-----+
|  Ham LLVM IR (Frontend)  |
+-----+
      |
      v
+-----+
|  Mem2Reg Pass (alloca->SSA)|
+-----+
      |
      v
+-----+
| Constant Folding & InstCombine |
+-----+
      |
      v
+-----+
|  CSE & Dead Code Elimination|
+-----+
      |
      v
+-----+
|  Loop Unrolling Pass      |
+-----+
      |
      v
+-----+
| Optimize Edilmiş LLVM IR  |
+-----+

```


3.2 New Pass Manager (NPM) vs. Legacy Pass Manager

LLVM 13 ve sonrasında eski (Legacy) Pass Manager tamamen kaldırılmış ve yerini **New Pass Manager (NPM)** almıştır. New Pass Manager'ın getirdiği temel yenilikler ve avantajlar şunlardır:

- Tür Güvenliği ve Derleme Zamanı Şablonları (Templates & CRTP):** Kalıtım (vtable) yerine C++ şablonları kullanılarak fonksiyon çağrı maliyetleri düşürülmüştür.
- Esnek Analiz Önbellekleme (Analysis Management):** Bir optimizasyon pass'i IR'ı değiştirmediyse, önceki analiz sonuçları (örn: Dominator Tree) tekrar hesaplanmaz.
- Piyasa Standartları Pipeline Desteği:** 01 , 02 , 03 , 0s , 0z gibi hazır optimizasyon seviyeleri kolayca oluşturulabilir.

NPM vs Legacy Performans Farkı

New Pass Manager, şablon tabanlı yapısı ve akıllı analiz önbellekleme (Analysis Caching) mekanizması sayesinde büyük projelerde derleme sürelerini %10 ile %25 arasında kısaltmaktadır.

3.3 Temel Optimizasyon Teknikleri ve IR Üzerinde İncelenmesi

Oyun motorlarında en yüksek kare hızlarına (FPS) ulaşmak için kritik öneme sahip 4 ana optimizasyon tekniğini inceleyelim:

1. Sabit Katlama (Constant Folding)

Derleme zamanında değeri bilinen sabit ifadelerin hesaplanarak tek bir değere indirgenmesidir.

• Optimize Edilmemiş IR:

```
%a = fadd float 10.0, 20.0
%b = fmul float %a, 2.0
```

• Constant Folding Sonrası IR:

```
; Derleyici (10.0 + 20.0) * 2.0 = 60.0 işlemini derleme zamanında yapar!
%b = store float 60.0
```

2. Ortak Alt İfade Eleme (Common Subexpression Elimination - CSE)

Program içinde birden fazla kez tekrarlanan aynı matematiksel hesaplamaları tespit edip tek bir hesaplama sonucunu tekrar kullanmaktır. Oyun içi vektör uzaklık hesaplarında hayati önem taşır.

• Ham IR:

```
%dx1 = fsub float %x2, %x1
%dy1 = fsub float %y2, %y1
; Tekrar eden aynı ifade:
%dx2 = fsub float %x2, %x1
```

- CSE Sonrası IR:

```
%dx1 = fsub float %x2, %x1
%dy1 = fsub float %y2, %y1
; %dx2 ifadesi silindi, %dx1 yazmacı doğrudan kullanıldı
```

3. Ölü Kod Eleme (Dead Code Elimination - DCE)

Programın çıktısını veya durumunu etkilemeyen, sonucu hiçbir yerde kullanılmayan (unreachable / unused) talimatların silinmesidir.

- Ham IR:

```
%unused_calc = fmul float %enemy_hp, 0.0 ; Sonucu hiçbir yerde kullanılmıyor
ret void
```

- DCE Sonrası IR:

```
ret void ; %unused_calc tamamen temizlendi!
```

4. Döngü Açma (Loop Unrolling)

Döngü dallanma maliyetini (branch prediction ve sayaç artırma) azaltmak için döngü gövdesini ardışık olarak genişletmektir. Oyun motorlarında matris çarpımlarında ve SIMD vektörizasyonunda sıklıkla uygulanır.

3.4 Özel Bir LLVM Pass Yazımı (Modern C++ NPM)

Şimdi oyun derleyicimiz için custom bir LLVM Pass yazalım. Bu Pass, oyun kodumuzdaki tüm yavaş bölme işlemlerini (`fdiv`) çarpmaya (`fmul`) dönüştüren bir optimizasyon yapsın.

C++ Kodu (src/custom_pass.cpp)

```

#include <llvm/IR/PassManager.h>
#include <llvm/IR/Instructions.h>
#include <llvm/IR/Constants.h>
#include <llvm/IR/IRBuilder.h>
#include <llvm/Support/raw_ostream.h>

// New Pass Manager için Custom Pass Sınıfı
class GameFastMathPass : public llvm::PassInfoMixin<GameFastMathPass> {
public:
    llvm::PreservedAnalyses run(llvm::Function& F, llvm::FunctionAnalysisManager& FAM) {
        bool changed = false;
        llvm::outs() << "GameFastMathPass Calisiyor: Fonksiyon " << F.getName() << "\n";

        for (auto& BB : F) {
            for (auto InstIt = BB.begin(); InstIt != BB.end(); ) {
                llvm::Instruction& I = *InstIt++;

                // Eğer bir FDiv (Float Bölme) işlemiyse ve bölen sabit bir sayıysa
                if (auto* fdiv = llvm::dyn_cast<llvm::BinaryOperator>(&I)) {
                    if (fdiv->getOpcode() == llvm::Instruction::FDiv) {
                        if (auto* c = llvm::dyn_cast<llvm::ConstantFP>(fdiv->getOperand(1))) {
                            // 1.0 / Constant hesabı derleme zamanında yap
                            double val = c->getValueAPF().convertToDouble();
                            if (val != 0.0) {
                                double recip = 1.0 / val;
                                llvm::IRBuilder<> builder(fdiv);
                                llvm::Value* recipVal = builder.getFloatTy()->isFloatTy() ?
                                    (llvm::Value*)llvm::ConstantFP::get(builder.getFloatTy(), recip) :
                                    (llvm::Value*)llvm::ConstantFP::get(builder.getDoubleTy(), recip);

                                // FDiv yerine FMul oluştur
                                llvm::Value* newMul = builder.CreateFMul(fdiv->getOperand(0), recipVal);
                                fdiv->replaceAllUsesWith(newMul);
                                fdiv->eraseFromParent();
                                changed = true;
                            }
                        }
                    }
                    InstIt++;
                }
            }
        }

        // Eğer IR değiştiyse analizleri invalidate et, değişmediyse koru
        return changed ? llvm::PreservedAnalyses::none() : llvm::PreservedAnalyses::all();
    }
};

```

İpucu: eraseFromParent() Kullanımı

Bir LLVM talimatını silmeden önce (`eraseFromParent()`), onun ürettiği sonucu kullanan diğer tüm talimatların `replaceAllUsesWith()` ile yeni talimata yönlendirildiğinden emin olunmalıdır!

3.5 Pass Pipeline Oluşturma ve Optimizasyon Çalıştırma

Oyun derleyicimizde bu custom pass'ı ve LLVM'in standart optimizasyon pipeline'ını nasıl çalıştıracağımızı görelim:

```
#include <llvm/Passes/PassBuilder.h>
#include <llvm/IR/PassManager.h>

void runOptimizationPipeline(llvm::Module& module) {
    llvm::LoopAnalysisManager LAM;
    llvm::FunctionAnalysisManager FAM;
    llvm::CGSCCAnalysisManager CGAM;
    llvm::ModuleAnalysisManager MAM;

    llvm::PassBuilder PB;

    // Analiz yöneticilerini kaydet
    PB.registerModuleAnalyses(MAM);
    PB.registerFunctionAnalyses(FAM);
    PB.registerRegisterClassBuilders(CGAM, LAM, FAM, MAM);
    PB.crossRegisterProxies(LAM, FAM, CGAM, MAM);

    // Module Pass Manager
    llvm::ModulePassManager MPM;

    // Standart O2 Optimizasyon Pipeline'ını Yükle
    MPM = PB.buildPerModuleDefaultPipeline(llvm::OptimizationLevel::O2);

    // Pipeline'ı Modül Üzerinde Çalıştır
    MPM.run(module, MAM);
    llvm::outs() << "=== Modul Optimizasyonlari Tamamlandi ===\n";
}
```

Bu bölüm ile LLVM Pass Manager altyapısını, New Pass Manager'ın mimarisini ve custom optimizasyon yazmayı öğrendik.

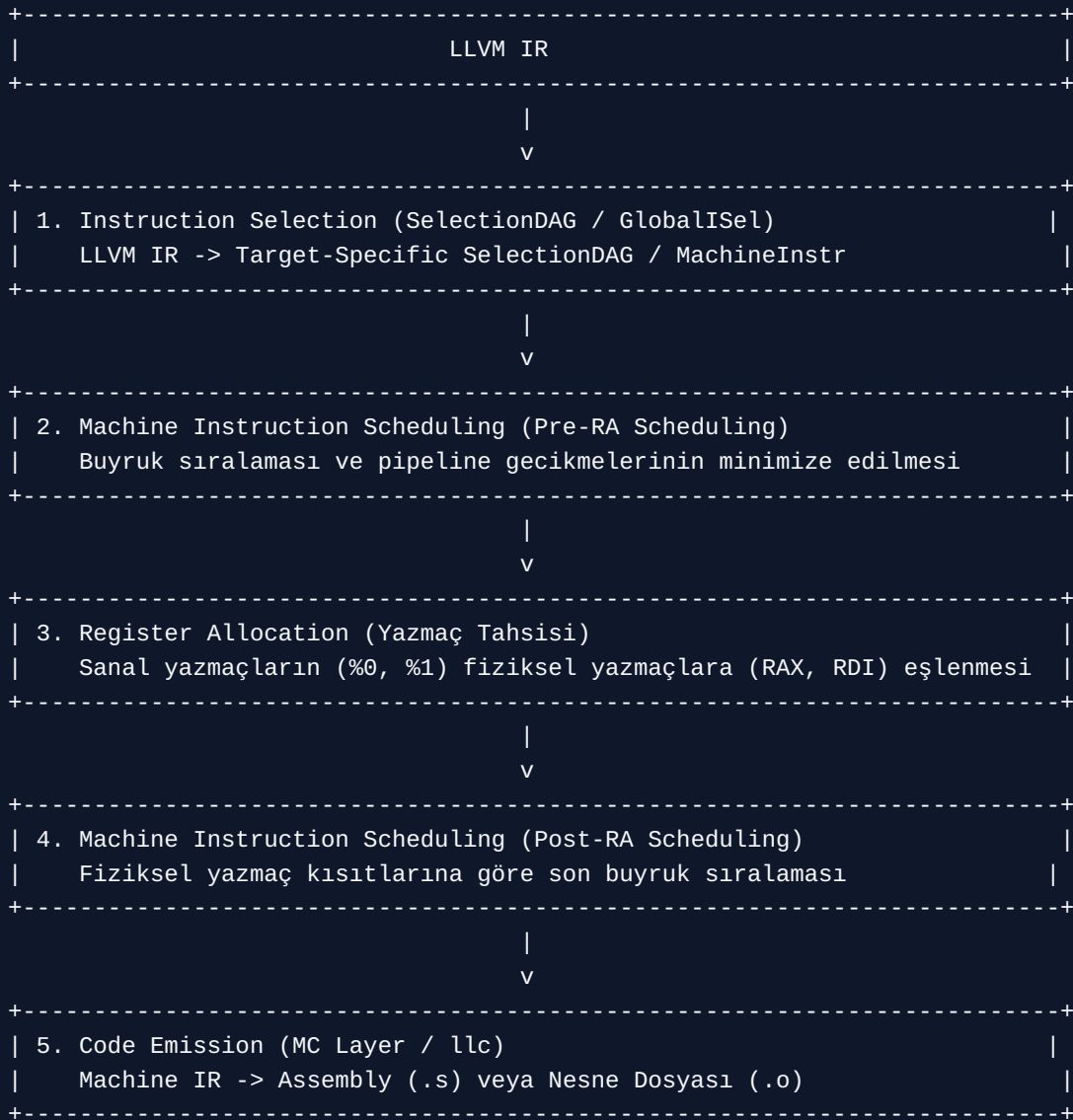
Bölüm 4: Backend Kod Üretimi, CodeGen

Mimarisi ve Hedef Makine (Target Machine)

4.1 LLVM Backend Boru Hattı (CodeGen Pipeline)

Middle-end katmanı tarafından optimize edilmiş LLVM IR, **Backend (Arka Yüz)** katmanına girdi olarak verilir. Backend'in görevi, bu soyut IR'ı hedef donanımın (x86_64, ARM64, RISC-V, WebAssembly vb.) mimarisine özel makine komutlarına ve ikili dosyalara (.o / .obj) dönüştürmektir.

LLVM Backend boru hattı 5 ana evreden oluşur:



Backend Esnekliği

Geliştirdiğiniz oyun dili için tek bir satır dahi C++ backend kodu yazmadan, sadece LLVM IR üreterek x86_64, ARM64 (Apple Silicon, Android), WebAssembly ve RISC-V mimarilerine kod çıktısı alabilirsiniz!

4.2 TableGen (.td) Dili ve Donanım Tanımlama

LLVM Backend'lerin en yenilikçi taraflarından biri **TableGen** altyapısıdır. Hedef işlemcinin yazmaçları, komut seti (instruction set), adresleme modları ve boru hattı (pipeline) özellikleri C++ koda elle sabitlenmek yerine `.td` uzantılı TableGen dosyalarında beyan edilir.

`llvm-tblgen` aracı bu `.td` dosyalarını okuyarak backend derlemesi sırasında otomatik C++ kodları üretir.

Örnek TableGen Tanımı (`TargetRegister.td` konsepti)

```
// x86_64 Akümülatör Yazmacı Tanımı
def RAX : Register<"rax">;
def RBX : Register<"rbx">;

// Donanım Komutu Tanımı: ADD komutu
class x86_instruction<bits<8> opcode, string asmstr> : Instruction {
  let OutOperandList = (outs GR64:$dst);
  let InOperandList = (ins GR64:$src1, GR64:$src2);
  let AsmString = asmstr;
}

def ADD64rr : x86_instruction<0x01, "addq $src2, $dst">;
```

4.3 Buyruk Seçimi (Instruction Selection): SelectionDAG vs. GlobalSel

Buyruk Seçimi, LLVM IR talimatlarını hedef işlemcinin desteklediği somut makine komutlarıyla eşleme işlemidir.

1. SelectionDAG (Geleneksel Yöntem):

- LLVM IR'ı yönlü devirsiz grafiklere (Directed Acyclic Graph - DAG) dönüştürür.
- Düğümeleri donanım komut modelleriyle eşleştirir (Pattern Matching).
- Yüksek optimizasyonlu kod üretir ancak hafıza kullanımı fazla ve derleme süresi daha uzundur.

5. GlobalSel (Global Instruction Selection - Modern Yöntem):

6. Fonksiyonun tamamı üzerinde DAG oluşturmadan Doğrudan Machine IR (MIR) seviyesinde çalışır.
7. Hızlı derleme süreleri sağlar (JIT ve debug derlemeleri için idealdir).

4.4 Yazmaç Tahsisi (Register Allocation) ve Scheduling

LLVM IR sonsuz sayıda **sanal yazmaç (virtual register)** kullanabilir (`%1`, `%2`, `%3` ...). Ancak gerçek işlemcilerin kısıtlı sayıda **fiziksel yazmaç (physical register)** vardır (örn: x86_64 için 16 genel amaçlı yazmaç).

Register Allocation (RA)

Register Allocator, sanal yazmaçları fiziki yazmaçlara atar. Eğer fiziki yazmaç sayısı yetersiz kalırsa, bazı değerleri yığına saklar ve tekrar okur. Bu duruma **Spilling** adı verilir.

Instruction Scheduling

İşlemci boru hattında (pipeline stalls) veri bağımlılıklarından kaynaklanan beklemleri önlemek için komutların sırasını değiştirir.

Spilling Maliyeti

Register Spilling, işlemci yazmaç yerine RAM/L1 Cache erişimi gerektirdiği için oyun gibi sıkı performans gerektiren döngülerde yavaşlamaya neden olur. LLVM Register Allocator bunu minimize edecek algoritmalar (Greedy Register Allocator) kullanır.

4.5 Machine IR (MIR) ve `llc` Aracı

Machine IR (MIR), LLVM IR'ın hedef mimariye özgü komutlarla temsil edilmiş halidir. `.mir` uzantılı dosyalar halinde diske aktarılabilir ve `llc` (LLVM Static Compiler) aracı ile doğrudan incelenebilir.

```
# LLVM IR dosyasını x86_64 Assembly koduna dönüştürme
llc -march=x86-64 -O3 input.ll -o output.s

# LLVM IR dosyasını doğrudan Nesne Dosyasına (.o) dönüştürme
llc -filetype=obj -march=x86-64 input.ll -o output.o
```

4.6 C++ API'si ile TargetMachine Yapılandırması ve Nesne Kod Üretimi (`.o`)

Oyun derleyicimizin en son aşamasında, ürettiğimiz LLVM modülünü diskte `.o` (Object File) olarak kaydedecek C++ kodunu yazalım.

C++ Kodu (`src/backend_codegen.cpp`)

```

#include <llvm/IR/Module.h>
#include <llvm/Support/TargetSelect.h>
#include <llvm/Support/TargetRegistry.h>
#include <llvm/Target/TargetMachine.h>
#include <llvm/Target/TargetOptions.h>
#include <llvm/Support/FileSystem.h>
#include <llvm/Support/raw_ostream.h>
#include <llvm/MC/TargetRegistry.h>
#include <llvm/IR/LegacyPassManager.h>
#include <system_error>

bool emitObjectFile(llvm::Module& module, const std::string& outputFilename) {
    // 1. Tüm Hedef Mimarileri İklendir
    llvm::InitializeAllTargetInfos();
    llvm::InitializeAllTargets();
    llvm::InitializeAllTargetMCs();
    llvm::InitializeAllAsmParsers();
    llvm::InitializeAllAsmPrinters();

    // 2. Varsayılan Hedef Üçlüsünü (Target Triple) Al (Örn: x86_64-pc-linux-gnu)
    auto targetTriple = llvm::sys::getDefaultTargetTriple();
    module.setTargetTriple(targetTriple);

    std::string error;
    auto target = llvm::TargetRegistry::lookupTarget(targetTriple, error);

    if (!target) {
        llvm::errs() << "HATA: Target bulunamadi: " << error << "\n";
        return false;
    }

    // 3. Hedef Makine Yapılandırması (CPU: generic, Features: none)
    auto cpu = "generic";
    auto features = "";

    llvm::TargetOptions opt;
    auto rm = std::optional<llvm::Reloc::Model>();
    auto targetMachine = target->createTargetMachine(targetTriple, cpu, features, opt, rm);

    module.setDataLayout(targetMachine->createDataLayout());

    // 4. Çıktı Dosyasını Açma (.o)
    std::error_code ec;
    llvm::raw_fd_ostream dest(outputFilename, ec, llvm::sys::fs::OF_None);

    if (ec) {
        llvm::errs() << "HATA: Çıktı dosyası açilamadı: " << ec.message() << "\n";
        return false;
    }

    // 5. CodeGen Pass Manager ile Nesne Dosyası Üretimi
    llvm::legacy::PassManager pass;
    auto fileType = llvm::CodeGenFileType::CGFT_ObjectFile;

```

```
if (targetMachine->addPassesToEmitFile(pass, dest, nullptr, fileType)) {  
    llvm::errs() << "HATA: TargetMachine bu dosya tipini üretemiyor!\n";  
    return false;  
}  
  
pass.run(module);  
dest.flush();  
  
llvm::outs() << "BAŞARILI: Nesne dosyası üretildi -> " << outputFilename << "\n";  
return true;  
}
```

Koda Adım Adım Bakış ve Açıklaması:

1. **InitializeAllTargets()** : LLVM static kütüphanelerinin hedef mimari sürücülerini (x86, ARM, AArch64, MIPS vb.) C++ runtime'a kaydeder.
2. **getDefaultTargetTriple()** : Kodun çalıştığı bilgisayarın işletim sistemi ve işlemci mimarisini belirler (örn: `x86_64-unknown-linux-gnu`).
3. **createTargetMachine()** : Hedef mimarinin bellek düzenini (Data Layout) ve komut setlerini barındıran TargetMachine nesnesini inşa eder.
4. **addPassesToEmitFile()** : LLVM IR'ı nihai makine koduna dönüştürüp akışa yazan arka yüz pass'lerini kurar.

Bölüm 5: İleri Düzey Konular: ORC JIT, MLIR, Shader Kod Üretimi ve Derleyici Araçları

5.1 LLVM ORC JIT (On-Request Compilation) Mimarisi ve Oyun İçi Betik (Scripting) Çalıştırma

Oyun motorlarında canlı kod yenileme (Hot-Reloading) ve hızlı betik (scripting) çalıştırma kritik gereksinimlerdir. Geleneksel olarak Lua veya C# gibi diller yorumlanarak (interpreted) veya sanal makine üzerinde çalıştırılır. LLVM **ORC JIT (On-Request Compilation)** API'si sayesinde, oyun içinde yazılan betik kodları çalışma zamanında milisaniyeler içinde doğrudan **özel makine koduna** derlenir ve sıfır performans kaybı ile çalıştırılır.

ORC JIT mimarisinin ana bileşenleri şunlardır: 1. **ExecutionSession** : Tüm JIT durumunu, thread'leri ve sembol tablolarını yönetir. 2. **JITDyLib** : Dinamik kütüphane benzeri sembol konteyneridir. Oyun motorunuzdaki C++ fonksiyonlarını JIT tarafına kaydetmenizi sağlar. 3. **JITLink** : Bellek içi nesne dosyalarını bağlayan ve sayfa izinlerini (Executable/Writable) ayarlayan ultra hızlı bağlayıcıdır.



JIT Scripting Avantajı

Oyun içi düşman yapay zekası (AI) veya fizik hesapları JIT ile derlendiğinde, yorumlanan Lua koduna kıyasla 10 kat ile 50 kat arasında performans artışı elde edilir.

C++ Kodu: Çalışma Zamanında Betik Çalıştıran JIT Motoru (`src/jit_engine.cpp`)

```

#include <llvm/ExecutionEngine/Orc/LLJIT.h>
#include <llvm/IR/LLVMContext.h>
#include <llvm/IR/Module.h>
#include <llvm/IR/IRBuilder.h>
#include <llvm/Support/InitLLVM.h>
#include <llvm/Support/TargetSelect.h>
#include <llvm/Support/raw_ostream.h>
#include <iostream>

// Oyun İçi Script Tarafından Çağrılacak C++ Fonksiyonu (Host Engine Function)
extern "C" void GameEngine_LogMessage(const char* msg) {
    std::cout << "[Oyun Motoru JIT Log]: " << msg << std::endl;
}

void runJITScript() {
    // 1. LLVM Target Sistemlerini Başlat
    llvm::InitializeNativeTarget();
    llvm::InitializeNativeTargetAsmPrinter();

    // 2. LLJIT Örneği Oluşturma
    auto jitOrErr = llvm::orc::LLJITBuilder().create();
    if (!jitOrErr) {
        llvm::errs() << "JIT Oluşturulamadı!\n";
        return;
    }
    auto jit = std::move(*jitOrErr);

    // 3. Thread-Safe LLVM Modülü ve Context Yapılandırması
    auto context = std::make_unique<llvm::LLVMContext>();
    auto module = std::make_unique<llvm::Module>("JITScriptModule", *context);
    llvm::IRBuilder<> builder(*context);

    // 4. C++ Tarafındaki Log Message Fonksiyonunu JIT Modülüne Tanımlama
    auto* logFuncType = llvm::FunctionType::get(builder.getVoidTy(), {builder.getPtrTy(), false});
    auto* logFunc = llvm::Function::Create(logFuncType, llvm::Function::ExternalLinkage, "GameEng

    // 5. JIT Script Fonksiyonu Oluşturma: void RunScript()
    auto* scriptFuncType = llvm::FunctionType::get(builder.getVoidTy(), false);
    auto* scriptFunc = llvm::Function::Create(scriptFuncType, llvm::Function::ExternalLinkage,

    auto* bb = llvm::BasicBlock::Create(*context, "entry", scriptFunc);
    builder.SetInsertPoint(bb);

    // Metin Sabiti Oluşturma
    auto* strVal = builder.CreateGlobalString("LLVM ORC JIT Script Basariyla Calisti!", "script_s

    builder.CreateCall(logFunc, {strVal});
    builder.CreateRetVoid();

    // 6. Modülü JIT'e Ekleme
    auto tsm = llvm::orc::ThreadSafeModule(std::move(module), std::move(context));
    cantFail(jit->addIRModule(std::move(tsm)));

    // 7. JIT İçindeki "RunScript" Sembolünü Bulma ve Çalıştırma
    auto symOrErr = jit->lookup("RunScript");

```

```

if (!symOrErr) {
    llvm::errs() << "JIT Sembolü Bulunamadı!\n";
    return;
}

// Sembolü C++ Fonksiyon Göstericisine (Function Pointer) Cast Etme
void (*scriptFn()) = symOrErr->toPtr<void (*)>();

// 8. KODU YEREL HIZDA ÇALIŞTIR!
scriptFn();
}

```

5.2 MLIR (Multi-Level Intermediate Representation) ve Oyun Motorları İçin Önemi

LLVM IR harika bir alt seviye temsil olsa da, yüksek seviyeli matematiksel yapılar (matris çarpımları, GPU thread blokları, oyun içi fizik simülasyonları) LLVM IR seviyesine indirildiğinde kaybolur.

MLIR (Multi-Level Intermediate Representation), LLVM projesinin çok seviyeli ara temsil altyapısıdır. **Dialect (Diyalekt)** adı verilen modüler yapıları sayesinde: * **Affine Dialect**: Döngü optimizasyonları ve bellek erişim desenleri için. * **Vector / Linalg Dialect**: Matrix/Vector işlemleri ve SIMD vektörizasyonu için. * **GPU Dialect**: CUDA, ROCm ve Vulkan Compute Kernel yönetimi için.

Oyun dilinizde önce MLIR seviyesinde matris ve vektör optimizasyonları yapabilir, ardından kodu LLVM IR'a indirgeyerek (lowering) donanıma gönderebilirsiniz.

5.3 LLVM IR'dan SPIR-V Üretimi ve DXC (DirectX Shader Compiler)

Modern oyun motorları grafik kartlarında (GPU) paralel hesaplama yapmak için SPIR-V (Vulkan) ve DXIL (DirectX 12) formatlarına ihtiyaç duyar.

- **LLVM SPIR-V Target / Backend**: Oyun dilinizden ürettiğiniz LLVM IR'ı `llvm-spirv` aracı veya kütüphanesi aracılığıyla doğrudan SPIR-V ikili formatına çevirebilirsiniz. Bu sayede hem CPU hem GPU kodunu **aynı dille** yazabilirsiniz!
- **DXC (DirectX Shader Compiler)**: Microsoft'un açık kaynaklı derleyicisidir. LLVM tabanlıdır ve HLSL shader kodlarını SPIR-V veya DXIL formatına dönüştürür.

5.4 Clang Tooling, LLD ve Clang Sanitizer Araçları

Bir oyun programlama dili ekosistemi geliştirmek yalnızca derleyici yazmaktan ibaret değildir; geliştirici araçlarına (Developer Tooling) da ihtiyaç vardır.

1. **Clang Tooling:** Oyun diliniz için otomatik kod biçimlendirici (Formatter), statik analiz araçları (Linter) ve Otomatik Tamamlama (Language Server Protocol - LSP) yazmanızı sağlar.
2. **LLD (LLVM Linker):** Oyun projenizin bağımlılıklarını bağlarken MSVC `link.exe` veya GNU `ld` yerine LLD kullanarak bağlama (linking) sürelerini 5-10 kat kısaltabilirsiniz.
3. **Clang Sanitizer Araçları:** Oyun motorunuzda bellek sızıntılarını ve paralel izlek (multithreading) hatalarını yakalamak için LLVM Sanitize altyapısı entegre edilmelidir:
4. **ASan (AddressSanitizer):** Yığın ve bellek taşmalarını (buffer overflow, use-after-free) yakalar.
5. **TSan (ThreadSanitizer):** Veri yarışlarını (data race) tespit eder.
6. **MSan (MemorySanitizer):** İklendirilmemiş bellek okumalarını yakalar.
7. **UBSan (UndefinedBehaviorSanitizer):** Tanımsız davranışları (sıfıra bölme, tamsayı taşması) tespit eder.

Oyun Testlerinde Sanitizer Kullanımı

Oyun geliştirme esnasında Debug derlemelerini ASan ve TSan ile çalıştırmak, karmaşık oyun fiziği ve çok izlekli (multithreaded) iş dizisi (job system) hatalarını aylar öncesinden tespit etmenizi sağlar.

Bu bölüm ile JIT scripting, MLIR mimarisi, GPU Shader üretimi ve profesyonel Clang araçlarını kapsamlı olarak öğrendik.